shots and collects the measurement results. Finally, Line 13 displays the measurement statistics. Unlike the ideal simulator, which consistently yields *{'1': 1000}*, the noisy simulator produces probabilistic outcomes such as *{'0': 56, '1': 944}* or *{'0': 50, '1': 950}*. The exact counts vary between runs because errors occur probabilistically. These results help illustrate the imperfect behaviour of real quantum hardware, offering a practical way to analyse noise effects without requiring physical quantum device access.

Now, let us investigate how changing the error probability affects the behaviour of the simulator. To do this, we create two modified versions of *qiskit7.py*, namely, *qiskit7_mod1.py* and *qiskit7_mod2.py*, by replacing Line 8 with *error = pauli_error([("X", 0.25), ("I", 0.75)])* and *error = pauli_error([("X", 0.80), ("I", 0.20)])*, respectively. We shall now execute all three programs and compare their outputs, as shown in Figure 1. Before running the programs, ensure that the virtual environment *qiskit_env* is activated, just as we did in the previous articles. As the probability of introducing the Pauli-X error increases, the measurement statistics change accordingly. Since the original circuit already applies a Pauli-X gate, an additional Pauli-X error occasionally flips the qubit back to its original state $|0\rangle$. Consequently, increasing the error probability increases the number of measurements that produce 0, while the number of measurements yielding 1 decreases. This simple experiment clearly demonstrates how Qiskit's noise model

```
(qiskit_env) # python qiskit7.py
{'0': 56, '1': 944}
(qiskit_env) # python qiskit7.py
{'0': 50, '1': 950}
(qiskit_env) # python qiskit7_mod1.py
{'1': 774, '0': 226}
(qiskit_env) # python qiskit7_mod1.py
{'0': 259, '1': 741}
(qiskit_env) # python qiskit7_mod2.py
{'1': 220, '0': 780}
(qiskit_env) # python qiskit7_mod2.py
{'1': 216, '0': 784}
```

Figure 1: The output of the three programs with different error rates

can be used to imitate the behaviour of increasingly noisy NISQ-era quantum hardware.

Now comes an important question: How do we decide whether the error probability introduced into our simulator is realistic? Is there a way to benchmark this artificially introduced error against the behaviour of an actual quantum computer? The answer is a resounding 'Yes' and, thanks to the technology giant IBM, you can do it free of cost.

In this section, we shall execute a modified version of our original Pauli-X gate program *qiskit4.py* on an actual superconducting quantum computer made available through the IBM Quantum Platform. The first step is to visit the IBM Quantum Platform portal and create a free account. Fortunately, you do not need a credit card to register, and the free tier provides limited access to real quantum hardware, allowing you to gain hands-on experience with an actual quantum computer. This is an extraordinary opportunity, considering that operating such sophisticated quantum hardware is extremely expensive. Commercial access to quantum computing resources can cost tens of dollars per minute, making IBM's free offering an invaluable resource for students, educators, researchers, and quantum computing enthusiasts around the world.

When you sign in to your IBM Quantum Platform account, you can select the region in which your instance is located: *us-east* or *eu-de*. Note, however, that the Open Plan (free plan) is currently available only in the *us-east* (Washington, D.C.) region. Once you have signed in, the *Home* screen provides an option to generate an API key, as shown in Figure 2. You will be prompted to provide a name for the key, after which you can either download it as a JSON file or simply copy it to the clipboard. This API key is then used by the program *qiskit8.py*, shown below, which applies a Pauli-X gate to a single qubit, just like the program *qiskit4.py*. The only difference is that *qiskit4.py* was executed on a simulator, whereas *qiskit8.py* is executed on an actual superconducting quantum computer. Yes—at this very moment, somewhere in a quantum computing facility in the United States, your Python program instructs one of the most ingeniously engineered machines ever built to generate precisely controlled microwave pulses and perform a genuine quantum computation. How wonderful is that!

```
1. from qiskit import QuantumCircuit
2. from qiskit.transpiler.preset_
   passmanagers import generate_
   preset_pass_manager
3. from qiskit_ibm_runtime import
   QiskitRuntimeService, SamplerV2 as
   Sampler
4. API_KEY="PasteYourAPIKeyHere"
5. QiskitRuntimeService.save_account(
```

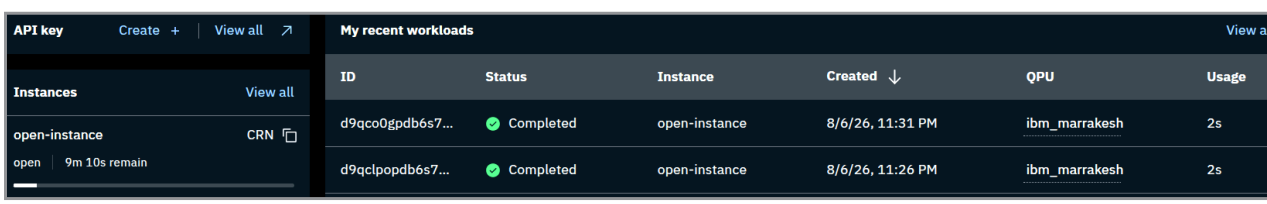

Figure 2: A portion of the IBM Quantum Platform dashboard